AI 開發全景入門

AI Coding Agent 實戰工作坊 — 初階場 (Codex)

目標： 了解 AI 輔助開發的完整生態，並且自己成功完成第一次 AI 協作

John Wu | Apr 2026

今天的旅程

時段	做什麼
觀念講解	先建立基本觀念
Live Demo	看 AI 怎麼一步步做事
練習時間	套到你自己的工作場景
回顧 Q&A	重點整理 + 課後練習說明

> 今天的重點是 **體驗一次完整流程**，不是把每個名詞背起來

Part 1: 觀念講解

什麼是 Vibe Coding？

傳統方式

你 → 想邏輯 → 寫 code → debug → 完成

Vibe Coding

你 → **描述意圖** → AI 生成 → **你驗收** → 完成

> 不再逐行手寫，而是用自然語言「說出你要什麼」

三個心智模型轉換

以前	現在
搜尋 — Google, Stack Overflow	**對話** — 直接問 AI，它給你完整答案
複製貼上 — 找到 code 改改看能不能用	**描述意圖** — 說你要什麼，AI 量身打造
獨自開發 — 自己想、自己寫	**人機協作** — 你做決策，AI 做執行

AI 工具

	Claude Code	Codex	Gemini CLI
公司	Anthropic	OpenAI	Google
專案指引檔	CLAUDE.md	AGENTS.md	GEMINI.md
延伸能力	Skills / MCP	Skills / MCP	Extensions / MCP
共通概念	都能有專案指引檔，也都能接外部工具與資料來源		

> 今天初階以 **Codex** 為主要工具

Prompts — 你當下怎麼開口

白話

你對 AI 說的每一句話，都是 Prompt

例子

「幫我在 `src/bookmark.ts` 加一個匯出功能」

> 描述越具體，AI 的結果越接近你要的

Project Instructions — 讓 AI 先了解你的專案

白話

在專案放一個專案指引檔，AI 自動讀取，不用每次重講

例子

`AGENTS.md`（Codex）裡面寫：
「這是 TypeScript 專案，用 Vitest 測試，commit 格式是 type(scope): description」

> 寫一次，全團隊受益 — AI 每次開工都知道規矩

Skills — 讓 AI 按照流程做事

白話

把常見工作流程寫成可重用的能力，讓 AI 在需要時套用

例子

寫好一個「TDD skill」後：

- 你可以明確指定它使用這個 skill
- 或在任務描述吻合時，讓 AI 自動套用

> 不用每次重複交代，寫一次就能重複使用

MCP — 讓 AI 連接外部工具與資料

白話

讓 AI 不只看你貼的內容，還能連到外部系統查資料、讀文件、做操作

例子

- 連 GitLab → 讀 Issue、MR
- 連 JIRA → 讀 Ticket
- 連 Telegram → 收發訊息

> AI 從「只會聊天」進一步變成「能查、能讀、能動手做事」

用「開餐廳」來比喻

你說的話	餐廳比喻	對應概念
「我想吃辣的」	先做一版試吃再調整	Vibe Coding
「宮保雞丁，不要花生，少油」	講越具體，出餐越準	Prompts
「這是川菜館，大火快炒」	廚房規矩寫好	Project Instructions
食譜 + 檢查點 + 工具清單	標準作業流程	Skill
接上供應商、倉庫、外送系統	對外接口	MCP

Context — 這些東西怎麼運作的？

每次你跟 AI 對話，工具通常會整理相關的 **Context** 給它，例如：

```
你的 prompt + 對話歷史 + 相關檔案內容 + 專案指引檔（如 AGENTS.md）+ 需要時載入的 Skills + 呼叫 MCP 後取得的結果
```

代價

- Context 越大 → 通常越慢、越貴
- 對話越長 → 前面不夠突出的重點，越可能被稀釋
- 帶太多不相關資訊 → 判斷品質會下降

> 所以：講重點、必要時開新對話、不要把什麼都塞進去

Models — AI 的法術強度

你可以先把不同模型想成遊戲裡不同等級的法術：

法術等級	感覺	代價	適合什麼時候用
小招級	最快、最省	🔥	整理文案、格式修正、簡單問答
主力技	速度和品質平衡	🔥🔥	日常 feature 開發、大部分工作
大招級	處理難題比較穩	🔥🔥🔥	卡很久的 bug、架構設計、複雜重構

> 不同工具可選的模型名稱會變，但挑選原則差不多：
> **簡單事別亂開大招，殺雞不用牛刀**

推理程度 — 法術蓄力

同一個法術，也可以想成有不同蓄力時間：

蓄力	感覺	代價	適合什麼時候用
低	幾乎秒放	🔥	很簡單的修改、格式整理
中	想一下再出手	🔥🔥	一般開發任務
高	想比較久，出手比較重	🔥🔥🔥	複雜邏輯、多步驟問題
超高	全力蓄力再丟大招	🔥🔥🔥🔥	困難 debug、架構設計、難拆的大任務

> 蓄力越久，通常答案會更完整；
> 但也會更慢、更貴。所以不是每次都要拉滿。

Token — AI 的魔力值

把 **Token** 想成 AI 處理資訊時的計量單位，也可以先理解成「魔力消耗單位」。

- 你講得越多，通常越耗
- AI 回得越多，通常越耗
- 對話越長、內容越亂，通常越耗

> 所以：講重點，少廢話，卡住就開新對話

魔力用完怎麼辦？

- 不同工具、不同方案，限制單位不太一樣
- 有些看 token，有些看訊息次數、使用額度或速率限制
- 有些方案會定期恢復
- 有些方案可以升級或加購，但不是每種都可以
- 強模型、長對話、重工具使用，通常都更耗資源

> 所以：簡單事用小招，難題再開大招
> 具體限制與恢復方式，還是要看工具的官方方案說明

初階你只要記得三件事

1. 先把需求講清楚
2. 再讓 AI 動手做
3. 最後一定要驗收結果

> 今天的成功標準不是懂所有工具，而是完成一次可驗收的協作

Part 2: Live Demo

Codex 畫面介紹

新增照片和檔案
規劃模式
Speed
GPT-5.4
本機
Standard
Default speed with normal credit usage
Fast
About 1.5x faster, with credits used at 2x
繼續
本機專案
剩餘的速率限制
5 小時 100% 凌晨12:51
1 週 94% 4月14日
如需取得更多存取權，請聯絡你的管理員。
深入了解
本機
預設權限
新增對話串
Search
技能和應用程式
自動化
對話串
live-demo
沒有對話串
database
terraform
handbook
sandbox-stack
設定
新增對話串
讓我們開始建立吧
live-demo
Build a classic Snake game in this repo.
Create a one-page $pdf that summarizes this app.
Create a plan to...
Ask Codex anything, @ to add files, / for commands, $ for skills
GPT-5.4
中
本機
預設權限
選取模型
GPT-5.4
GPT-5.4-Mini
GPT-5.3-Codex
GPT-5.2
選取推理
低
中
高
超高
上下文視窗：
已使用 14%（剩餘 86%）
已使用 35k / 258k 個自字元
Codex 會自動壓縮上下文

Demo 1 — Vibe Coding

一句話做出一個遊戲

1. 跟 Codex 說：**「幫我做一個打地鼠遊戲」**
2. 看到第一版後，給反饋迭代：
   - 「打到地鼠時加爆炸特效」
   - 「地鼠出現速度要越來越快」
3. 滿意後收工

Demo 2 — Prompts 的差異

同一個需求，不同的說法

	Prompt	預期結果
❌	「改好一點」	AI 不知道要改什麼
⚠️	「UI 醜醜的，幫我改」	AI 隨便改，可能不是你要的
✅	「把背景改成草地主題，地鼠換成卡通風格，按鈕加圓角和陰影，被打到時要有特效，游標變錘子」	AI 精準執行

> 現場用第三種 Prompt 讓 Codex 改造 Demo 1 的遊戲

Demo 3 — AGENTS.md

讓 AI 記住專案規矩

延續 Demo 1 的遊戲，建立 `AGENTS.md`：

```
# 打地鼠遊戲

## 技術棧
HTML + CSS + vanilla JavaScript，不用任何框架

## 規則
- 所有文字使用繁體中文
- CSS 使用卡通風格，圓角 + 明亮配色
- 每次修改後要能直接開 index.html 測試
```

> 之後每次跟 Codex 對話，它通常會先讀取並盡量遵守這些規矩

Demo 4 — Skills

讓 AI 按照流程做事

```
在專案中建立一個 `code-review` Skill：
每次要做收尾檢查時，執行：
1. 檢查有沒有 console.log 殘留
2. 確認所有文字都是繁體中文
3. 測試遊戲能不能正常開啟
4. 列出這次改了哪些檔案
```

> 之後你可以明確叫它跑這個 skill，或在任務吻合時讓 AI 套用這個流程

Demo 5 — Superpowers

整套技能包

1. 安裝 Superpowers
2. 直接跟 Codex 描述任務，必要時明確指定要用的 skill
   例：「請用 brainstorming 先收斂需求，再幫我加多人即時對戰模式」
3. 觀察它如何把工作拆成比較結構化的流程

觀察重點

- 不用自己一個個重寫常見 workflow
- AI 會依照安裝好的技能包，用比較穩定的方式做事

Demo 6 — OpenSpec

用 Spec 驅動開發

1. 安裝 OpenSpec，執行 `openspec init`
2. `/opsx:propose` — 建立需求提案
   （例：「`/opsx:propose` 幫打地鼠遊戲加難度選擇」）
3. `/opsx:apply` — 確認 spec 後讓 AI 實作
4. `/opsx:archive` — 驗收完成後歸檔

> 需求有記錄、變更有追蹤，AI 照 spec 做事而非靠模糊記憶

Demo 回顧 — 從隨便講講到有規矩有記錄

Demo	主題	你學到的
1	Vibe Coding	一句話做出來
2	Prompts	說清楚，做更好
3	AGENTS.md	規矩寫一次，永遠生效
4	Skills	流程自動化
5	Superpowers	整套技能包一鍵安裝
6	OpenSpec	需求變成可追蹤文件

Part 3: 練習時間 (20 min)

把今天的方法套到你自己的工作

⚠️ 練習前提醒：不要貼機密資料

- **不要貼** API key、密碼、token、客戶個資
- **不要貼** 未公開的核心演算法或商業策略
- 不確定能不能貼？→ 先用公開或模擬資料練習

> 詳細說明放在回顧段落，這裡先記住：**有疑慮就別貼**

自由實作

現在換你把剛剛的做法用在自己的工作上

- 不求做大，只求做完一個可驗收的小成果
- 重點不是一次做對，而是讓 AI 先做第一版，再由你迭代
- 今天的成功標準：**你知道怎麼開始、怎麼追問、怎麼驗收**

自由實作怎麼選題目

三個判準

- **小** — 單一功能、單一 bug、單一整理任務，不跨很多系統
- **清楚** — 你能用一句話說清楚要它做什麼
- **可驗收** — 最後能用測試、畫面、或 checklist 判斷結果

不適合的題目

- 「幫我重構整個後台」
- 「把整個系統改漂亮」
- 「一次做完整個新產品」

不知道怎麼開口？先用這個模板

工程師

「這是我的小任務：____。請你先做第一版，如果需要修改程式碼就直接改，最後告訴我怎麼驗收，或幫我跑相關測試。」

QA

「這是我要測的功能 / 這是我觀察到的問題：____。請幫我整理成 test checklist 或 bug report，格式要清楚、可直接拿去用。」

BA

「這是我的需求描述：____。請幫我整理成 user story、acceptance criteria，並指出哪裡還不夠清楚。」

卡住時怎麼接著問

常用追問

- 「測試失敗，錯誤如下，請修到通過：____」
- 「結果跟我預期不同，差異是：____」
- 「先不要一次做完，請拆成 3 個步驟」
- 「先告訴我你準備怎麼改，再開始動手」

完成標準

- [ ] AI 幫你產出第一版
- [ ] 你有給至少一次回饋或修正方向
- [ ] 最後有驗收依據，不是只看感覺

Part 4: 回顧

今天走過的全景地圖

```
✅ Vibe Coding   — 先做出來，再逐步修正
✅ Prompts       — 問得越清楚，答得越接近
✅ AGENTS.md     — 讓 AI 知道你的專案規矩
✅ Skills        — 讓 AI 按固定流程工作
✅ Superpowers   — 把常見流程包成可重用能力
✅ OpenSpec      — 把需求從對話變成可追蹤 spec
```

安全提醒：AI 不是保險箱

不要貼給 AI 的東西

類型	範例	風險
機密程式碼	核心演算法、未公開功能	把敏感資訊送進第三方服務
客戶資料	個資、交易紀錄	違反隱私法規
認證資訊	API key、密碼、token	外洩即可被利用
內部文件	未公開的商業策略	競爭資訊外洩

> 是否可使用，依公司政策、工具設定與資料分級規範為準

離開教室後你可以怎麼開始

回去就做這三件事

1. 找一個 **30 分鐘內能驗收** 的小任務
2. 用 AI 先做第一版，不要一開始就追求完美
3. 把錯誤、差異、限制繼續丟回去迭代

三個 Takeaway

1. AI 是你的 Pair Programmer

AI 是協作者，不是代做者；**你要負責驗收**

2. AI 不只靠 Prompts

不是越長越好，而是越清楚越好；先講 **目標、限制、驗收**

3. AI 會犯錯

卡住時不要自己硬修；把 **錯誤和預期差異** 丟回 AI

Q&A